

Indira Gandhi Delhi Technical University for Women



Computer Organization and Architecture

Lab File

Course Code: BCS 203

Submitted by: Aashi Jain

Submitted to: Ms. Shruty Ahuja

Enrollment number: 00701172024

Branch and Section: CSE-AI(1)

INDEX

Write a program to perform:

S.NO	TITLE	Page No.	Date	Remarks
01	To verify the truth tables of AND, OR, NOT, NAND, NOR, XOR and XNOR gates	3	29.08.25	
02	To verify that NAND and NOR gates are universal gates	7	05.09.025	
03	To design a Half Adder circuit and verify its truth table	10	05.09.025	
04	To design a Half Subtractor circuit and verify its truth table	11	05.09.025	
05	To design a Full Adder circuit and verify its truth table	12	12.09.025	
06	To design a Full Subtractor circuit and verify its truth table	14	12.09.025	
07	To design a 2-bit binary incrementer using half adder circuit	17	19.09.025	

PRACTICAL 01

Objective: To verify the truth tables of AND, OR, NOT, NAND, NOR, XOR and XNOR gates

Theory:

Logic Gates - Logic gates are the fundamental building blocks of digital circuits, designed to perform basic logical operations. Each logic gate processes binary inputs (0s and 1s) and produces a single binary output based on its specific function e.g. AND, OR, NAND, NOR, XOR etc.

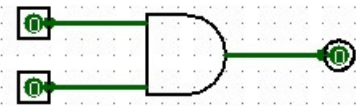
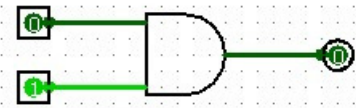
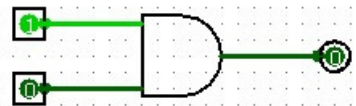
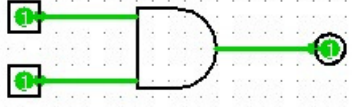
E.g. an AND gate outputs 1 only when all inputs are 1, whereas an OR gate outputs 1 if at least one input is 1. The NOT gate, on the other hand, inverts the input, outputting 1 for 0 and 0 for 1.

Truth Table - The behaviour of logic gates is best represented using a truth table, which systematically lists all possible input combinations and their corresponding outputs. Truth tables provide a clear and structured way to understand how a logic gate processes inputs, making them essential tools in designing and analysing digital systems.

1. AND

a. The AND gate is a logic gate that performs the logical conjunction operation. It takes two binary inputs and produces a single binary output. The output is 1 (true) only if both inputs are 1 (true); otherwise, the output is 0 (false). The operation follows the rule: where Y is the output, and A and B are the binary inputs. The AND gate is $Y = A \cdot B$ commonly used in digital circuits where multiple conditions need to be satisfied simultaneously.

b. Truth Table –

Input A	Input B	Output Y = A·B	Simulation
0	0	0	
0	1	0	
1	0	0	
1	1	1	

c. Applications –

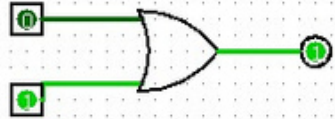
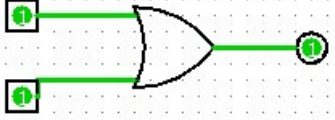
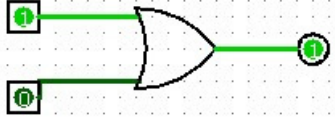
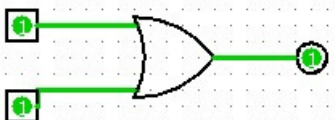
i. Traffic light control systems

- ii. Robotics control system
- iii. Gaming consoles
- iv. Safety mechanisms in machinery

2. OR

a. The OR gate is a logic gate that performs the logical disjunction operation. It takes two or more binary inputs and produces a single binary output. The output is 1 (true) if at least one of the inputs is 1 (true); otherwise, the output is 0 (false). The OR gate can be mathematically expressed as: where Y is the output, and A and B are the $Y=A+B$ binary inputs. OR gates are commonly used in circuits where one or more conditions need to be satisfied for a system to operate.

b. Truth Table -

Input A	Input B	Output $Y=A+B$	Simulation
0	0	0	
0	1	1	
1	0	1	
1	1	1	

c. Applications –

- i. Alarm systems
- ii. Fault detection systems
- iii. Volting mechanisms in circuits

3. NOT

a. The NOT gate, also known as an inverter, is a logic gate that performs the logical negation operation. It has a single input and a single output. The NOT gate inverts the input, producing an output of 1 (true) when the input is 0 (false) and vice versa. Mathematically, it is expressed as: where Y is the output, and A is the input. $Y=A$

b. Truth Table -

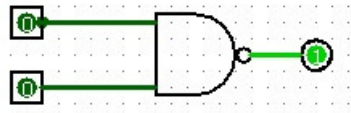
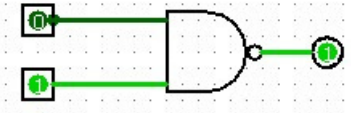
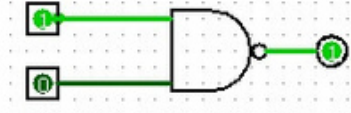
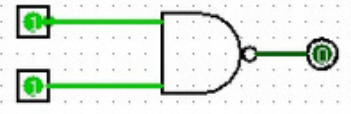
Input A	Input B	Simulation
0	1	

1	-0	
---	----	---

c. Applications -

- Signal inversion
 - Complement operations
 - Control signal operation
4. NAND a. The NAND gate is a universal logic gate that performs the inverse of the AND operation. It outputs 0 (false) only when all its inputs are 1 (true); otherwise, the output is 1 (true). Mathematically, it is expressed as: where Y is the output, and A and B are the $Y = A \cdot B$ inputs.

b. Truth Table –

Input A	Input B	Output Y= A.B	Simulation
0	0	1	
0	1	1	
1	0	1	
1	1	0	

c. Applications –

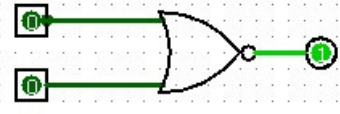
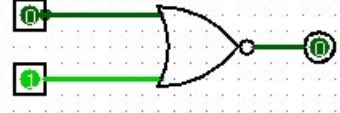
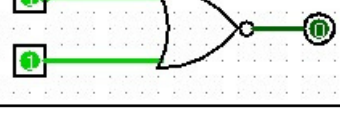
- Memory storage elements
- Flip-flop design
- Universal gate implementation

5. NOR

a. The NOR gate is another universal logic gate that performs the inverse of the OR operation. It outputs 1 (true) only when all its inputs are 0 (false); otherwise, the output is 0 (false). Mathematically, it is expressed as: where Y is the output, and A and B are the inputs.

$$Y = A + B$$

b. Truth Table -

Input A	Input B	Output Y= A.B	Simulation
0	0	1	
0	1	0	
1	0	0	
1	1	0	

c. Applications

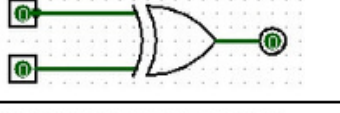
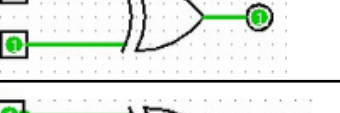
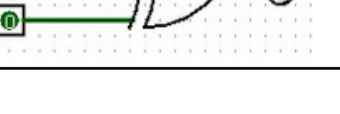
- Control logic circuits
- Universal gate design
- Digital circuit design

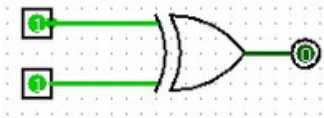
6. XOR

a. The XOR (exclusive OR) gate outputs 1 (true) only when the number of true inputs is odd, or specifically, when the inputs are different. It outputs 0 (false) if both inputs are the same. Mathematically, it is expressed as: where $Y = (A \cdot \bar{B}) + (\bar{A} \cdot B)$ Y is the output, and A and B are the inputs.

$$A \oplus B = A \bar{B} + \bar{A} B$$

b. Truth Table -

Input A	Input B	Output Y= A.B	Simulation
0	0	0	
0	1	1	
1	0	1	

1	1	0	
---	---	---	---

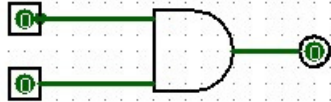
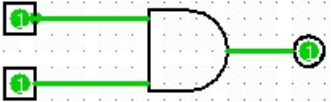
c. Applications –

- i. Error detections and corrections
- ii. Data encryption
- iii. Digital comparators

7. XNOR

a. The XNOR (exclusive NOR) gate is the complement of the XOR gate. It outputs 1 (true) if the inputs are the same (both 0 or both 1) and 0 (false) if the inputs are different. Mathematically, it is expressed as: where Y is the $Y = A \oplus B = (A \cdot B) + (\overline{A} \cdot \overline{B})$ output, and A and B are the binary inputs.

b. Truth Table -

Input A	Input B	Output Y= A.B	Simulation
0	0	1	
0	1	0	
1	0	0	
1	1	1	

c. Applications –

- i. Parity generators and checkers
- ii. Data integrity verification
- iii. Logic equality circuits

PRACTICAL 02

Objective: To verify that NAND and NOR gates are universal gates

Theory:

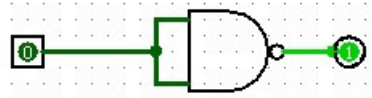
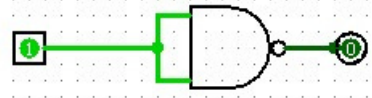
Universal Gates - Universal gates are logic gates that can be used to construct any other logic gate, such as AND, OR, NOT, XOR, and XNOR. The two universal gates are NAND and NOR. Their universality lies in their ability to perform all basic logical operations (AND, OR, NOT) and implement any Boolean function.

1. NAND Gate as Universal gate

a. For NOT Gate

i. A single NAND gate with both inputs connected to the same signal acts as a NOT gate. Mathematically, it is expressed as: where Y is the output, and A is $Y = A \cdot A$ the input.

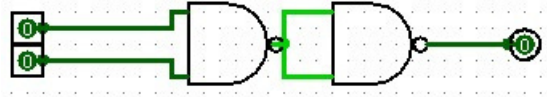
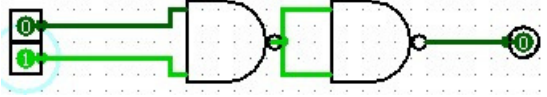
ii. Truth Table -

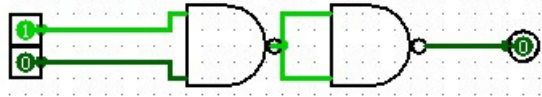
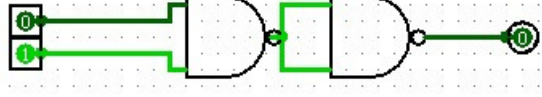
Input A	Output Y= A	Stimulation
0	1	
1	0	

b. For AND Gate

i. For AND gate, we use two NAND gates, first one to convert the input and then reverse the result using another NAND.

ii. Truth Table –

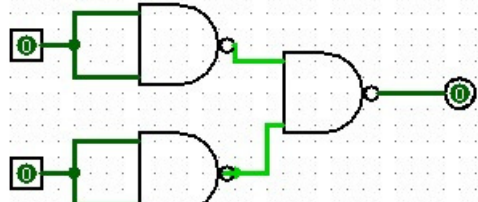
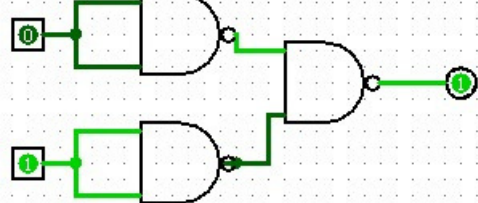
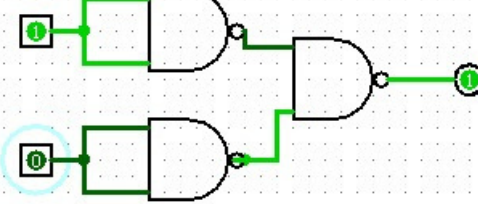
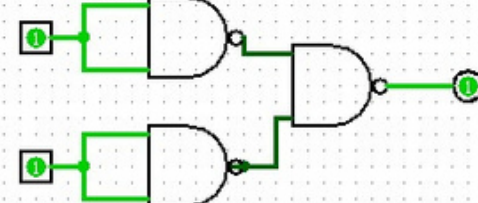
Input A	Input B	Output	Simulation
0	0	0	
0	1	0	

1	0	0	
1	1	1	

c. For OR Gate

i. For OR gate, we use three NAND gates, first one to convert the inputs and then apply NAND on the received inputs using another NAND gate.

ii. Truth Table -

Input A	Input B	Output	Simulation
0	0	0	
0	1	1	
1	0	1	
1	1	1	

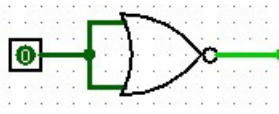
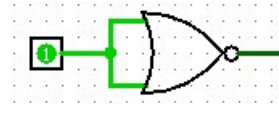
2. NOR Gate As Universal Gate

a. For NOT Gate

i. A single NOR gate with both inputs connected to the same signal acts as a NOT gate. Mathematically, it is expressed as: where Y is the output, and A is the input.

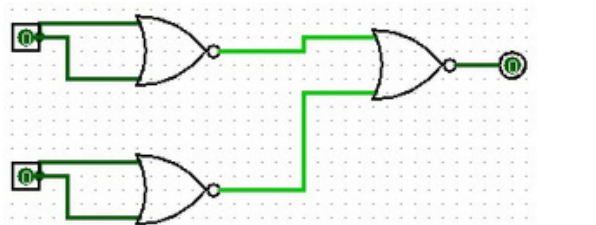
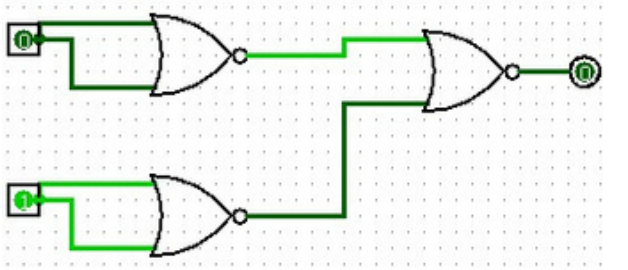
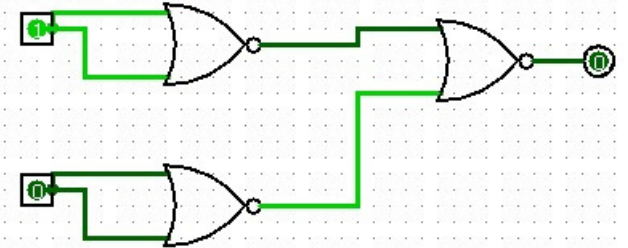
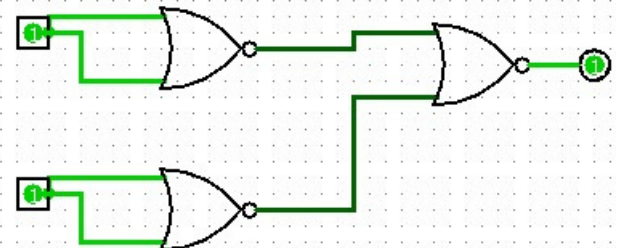
$$Y = A + A$$

ii. Truth Table –

Input A	Input B	Output	Simulation
0		1	
1		0	

c. For OR Gate i. For OR gate, we use two NOR gates, first one to convert the inputs and then apply NOR on the received inputs using another NOR gate.

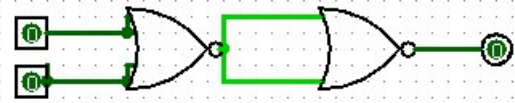
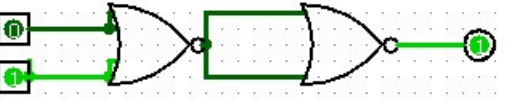
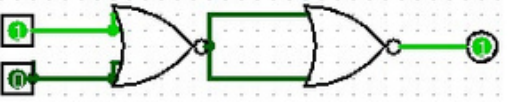
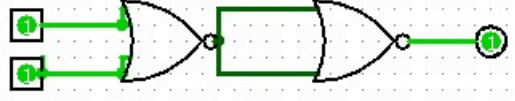
ii. Truth Table -

Input A	Input B	Output	Simulation
0	0	0	
0	1	0	
1	0	0	
1	1	1	

c. For OR Gate

i. For OR gate, we use two NOR gates, first one to convert the inputs and then apply NOR on the received inputs using another NOR gate

. ii. Truth Table -

Input	Input	Output	Simulation
0	0	0	
0	1	1	
1	0	1	
1	1	1	

PRACTICAL 03

Objective: To design a Half Adder circuit and verify its truth table. **Theory :** A Half Adder is a basic combinational logic circuit used for the binary addition of two single-bit numbers. It produces two outputs:

1. Sum (S) – Represents the sum of the two input bits.
2. Carry (C) – Represents the carry generated.

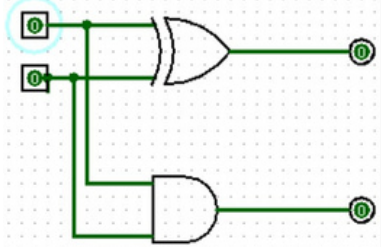
The Half Adder is a fundamental building block in digital electronics, especially in arithmetic and logic circuits.

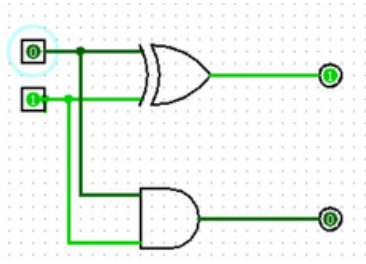
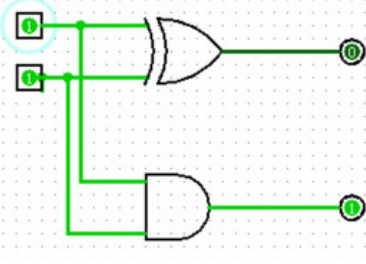
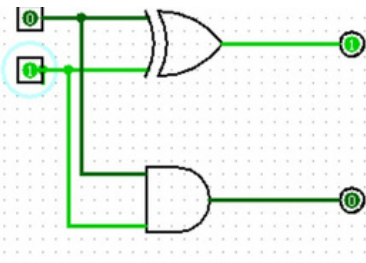
A Half Adder operates using two logic gates:

1. XOR Gate for the Sum output.
2. AND Gate for the Carry output.

The Boolean expressions for the outputs are:

1. Sum (S) = $A \oplus B$ - A XOR B
2. Carry (C) = $A \cdot B$ - A AND B

Input A	Input B	Sum	Carry	Simulation
0	0	0	0	

0	1	1	0	
1	1	1	1	
1	0	1	0	

Applications - 1. Used in binary addition circuits. 2. Forms the basic unit for building Full Adders and complex arithmetic circuits. 3. Used in ALUs (Arithmetic Logic Units) inside processors.

PRACTICAL 04

Objective : To design a Half Subtractor circuit and verify its truth table

Theory : A Half Subtractor is a digital logic circuit that performs binary subtraction of two single-bit binary numbers.

It produces two outputs:

1. Difference (D) – Represents the difference of the two input bits.
2. Borrow (B) – Represents the borrow generated.

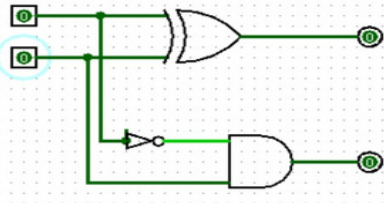
The Half Subtractor is a fundamental building block in digital electronics, especially in arithmetic and logic circuits.

A Half Subtractor operates using two logic gates:

1. XOR Gate for the Difference output.
2. AND Gate for the Borrow output.

The Boolean expressions for the outputs are:

1. Difference (D) = $A \oplus B$ - A XOR B
2. Borrow (B') = $A \cdot B$ - NOT A AND B

Input A	Input B	Difference	Borrow	Simulation
0	0			

0	1			
1	1			
1	0			

PRACTICAL 05

Objective: To design a Full Adder circuit and verify its truth table

Theory: A Full Adder is a combinational logic circuit used for binary addition of three input bits:

1. A - First input bit
2. B - Second input bit
3. Carry-in - Carry input from the previous stage

It produces two outputs:

1. Sum (S) – Represents the sum of the input bits.
2. Carry-out (Cout) – Represents the carry generated during addition.

A Full Adder can be constructed using two Half Adders and an OR gate.

The first Half Adder takes inputs A and B, producing an intermediate sum ($S_1 = A \oplus B$) and an intermediate carry ($C_1 = A \cdot B$).

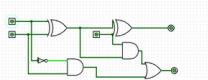
The second Half Adder then takes S_1 and the carry-in (C_{in}) as inputs, generating the final sum $S = S_1 \oplus C_{in}$ and an additional carry $C_2 = S_1 \cdot C_{in}$.

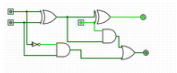
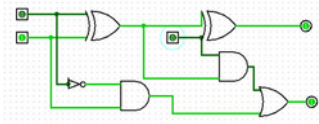
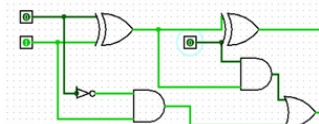
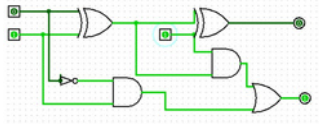
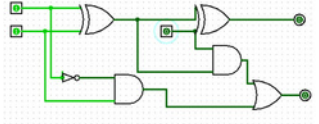
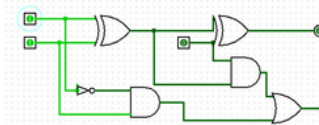
The final carry-out (C_{out}) is obtained by OR-ing C_1 and C_2 using an OR gate ($C_{out} = C_1 + C_2$).

This modular approach simplifies Full Adder design and allows easy extension to multi-bit adders.

The Full Adder overcomes the limitation of the Half Adder, which cannot handle carry propagation from previous additions.

Full Adders are essential building blocks in Arithmetic Logic Units (ALUs)

Input A	Input B	Carry in	Sum	Carry out	Simulation
0	0	0	0	0	

0	0	1	1	0	
0	1	0	1	0	
0	1	1	0	1	
1	0	0	1	0	
1	0	1	0	1	
1	1	0	0	1	
1	1	1	1	1	

PRACTICAL 06

Objective: To design a Full Subtractor circuit and verify its truth table

Theory:

A Full Subtractor is a combinational logic circuit designed to perform binary subtraction of three input bits:

1. A – Minuend bit
2. B – Subtrahend bit
3. Bin (Borrow-in) – Borrow input from the previous stage

It produces two outputs:

1. Difference (D) – Represents the subtraction result of the input bits.
2. Borrow-out (Bout) – Represents the borrow generated during subtraction.

A Full Subtractor can be constructed using two Half Subtractor and an OR gate.

- The first Half Subtractor takes inputs A and B, producing an intermediate difference ($D1 = A \oplus B$) and borrow ($B1 = A' \cdot B$).

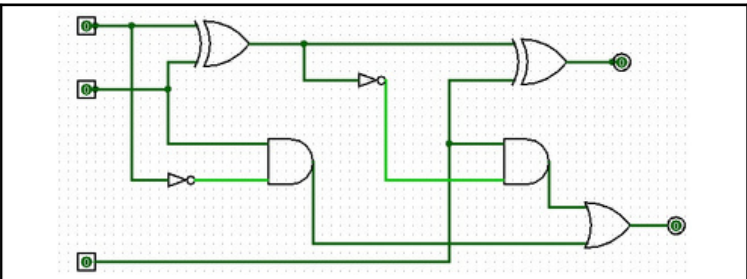
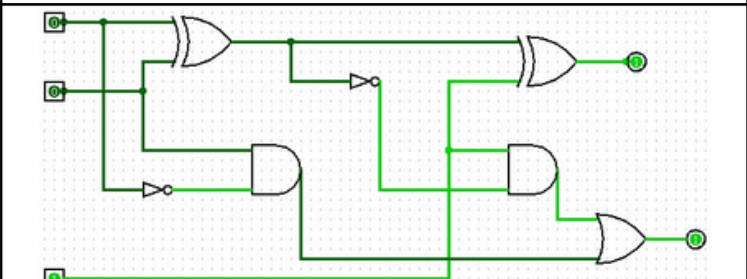
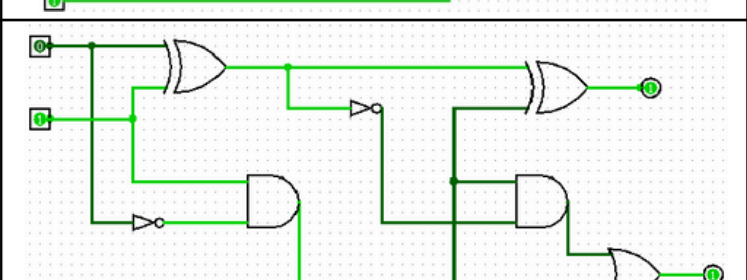
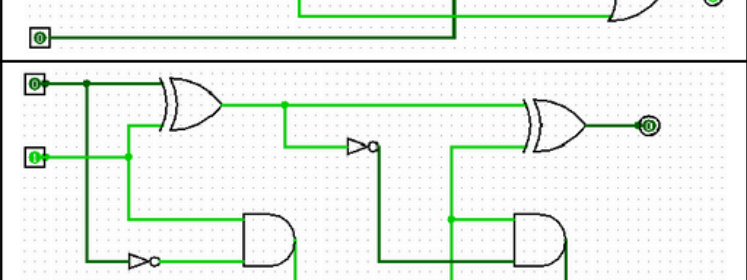
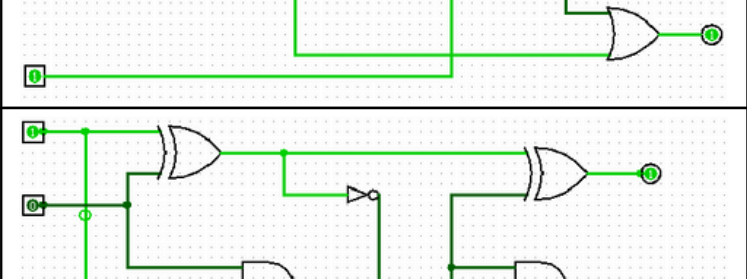
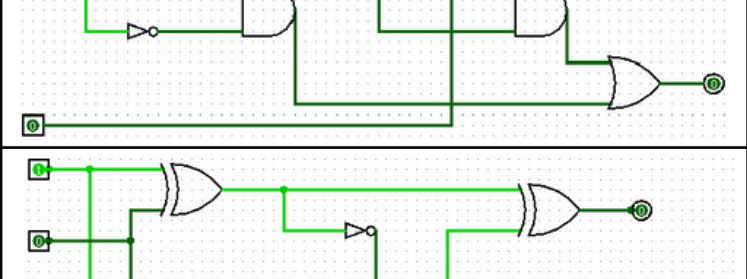
- The second Half Subtractor then takes D1 and Borrow-in (Bin) as inputs, generating the final difference $D = D1 \oplus Bin$ and an additional borrow $B2 = D1' \cdot Bin$

- The final Borrow-out (Bout) is obtained by OR-ing B1 and B2 using an OR gate ($Bout = B1 + B2$).

This modular approach allows the Full Subtractor to handle borrow propagation, which a Half Subtractor alone cannot do. Full Subtractor are essential components in digital circuits that perform binary subtraction, such as Arithmetic Logic Units (ALUs) and Subtractor circuits in processors. The logical expressions for the outputs are:

- Difference Output (D): $D = A \oplus B \oplus Bin$
- Borrow Output (Bout): $Bout = (A' \cdot B) + ((A \oplus B)' \cdot Bin)$

Input A	Input B	Borrow in	Difference	Borrow out	Stimulation

0	0	0	0	0	
0	0	1	1	1	
0	1	0	1	1	
0	1	1	0	1	
1	0	0	1	0	
1	0	1	0	0	

1	1	0	0	0	
1	1	1	1	1	

PRACTICAL 07

Objective: To design a 2-bit binary incrementer using half adder circuit

Theory:

A **2-bit binary incrementer** is a combinational logic circuit that increases a 2-bit binary number by one (1). The binary incrementer **adds '1'** to the existing 2-bit binary input, generating a 2-bit output along with a carry if required.

The operation of incrementing can be efficiently performed using **Half Adders**, where The Least Significant Bit (LSB) is directly added with **1** using the **first Half Adder**.

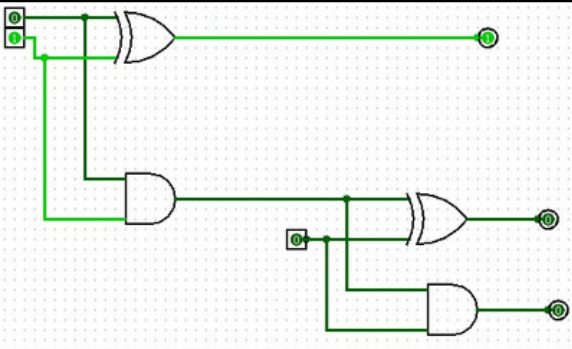
- The resulting carry from the first addition is then added to the next significant bit (MSB) using the **second Half Adder**.

The half adder takes two inputs and provides two outputs:

- **Sum** = $A \oplus B$ (XOR of inputs)
- **Carry** = $A \cdot B$ (AND of inputs)

In this design:

- The binary number is represented as A B (where A is MSB and B is LSB).
- The increment operation adds 1 to A.
- If A is 1, a carry is generated and passed to B, which is then incremented. This approach is simple and demonstrates how addition and incrementation operations are achieved using basic logic gates and Half Adder circuits. Binary incrementers are commonly used in counters, arithmetic units, and control systems in digital circuits.

A	B	Incremented Output	Carry-out	Stimulation
0	0	0	0	

0	1	1	0	
1	0	1	0	
1	1	0	1	